ALGORITMIA · PRÁCTICAS

Compilar

De tus ficheros `.c` al programa que se ejecuta. Y qué hacer cuando gcc se queja.

Un programa en C son varios ficheros

CABECERA

`pila.h`

Dice QUÉ hay: los tipos y los nombres de las funciones. No tiene código.

FUENTE

`pila.c`

Dice CÓMO se hace: el código de cada función de la librería.

FUENTE

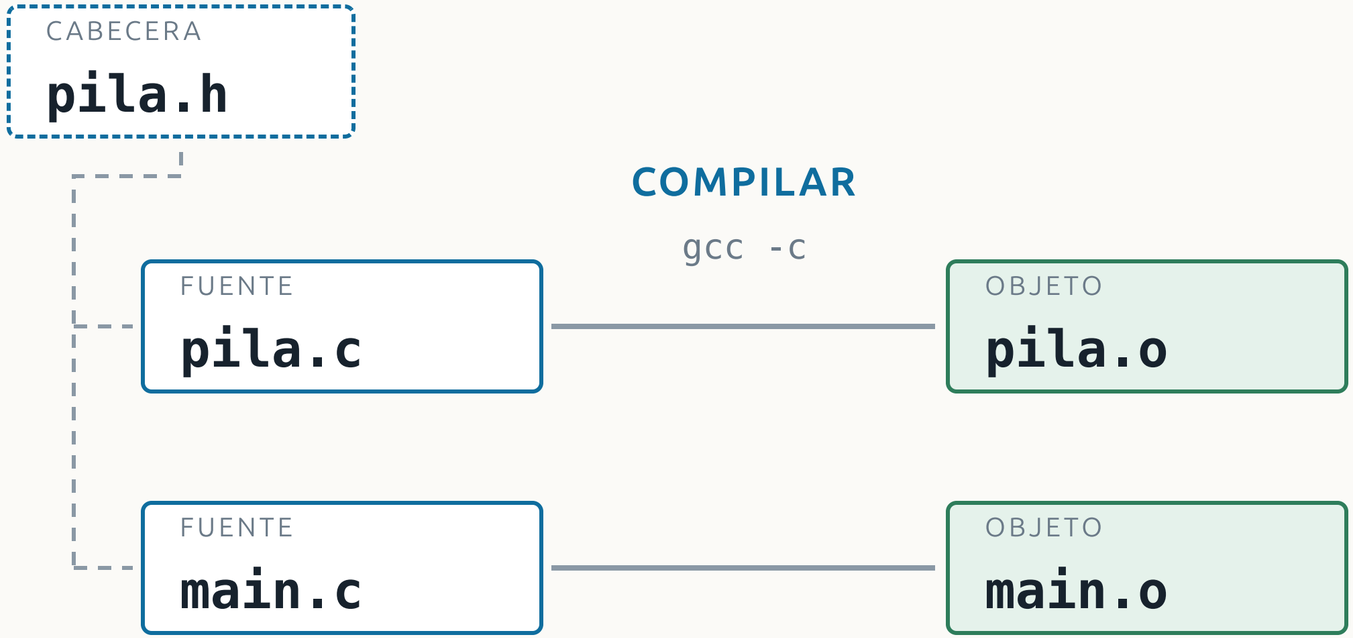
`main.c`

El programa que USA la librería. Aquí está el main.

Los tres son **texto**. Ninguno se puede ejecutar todavía.

Compilar es traducir

Cada fichero .c se traduce por separado, sin mirar los demás.



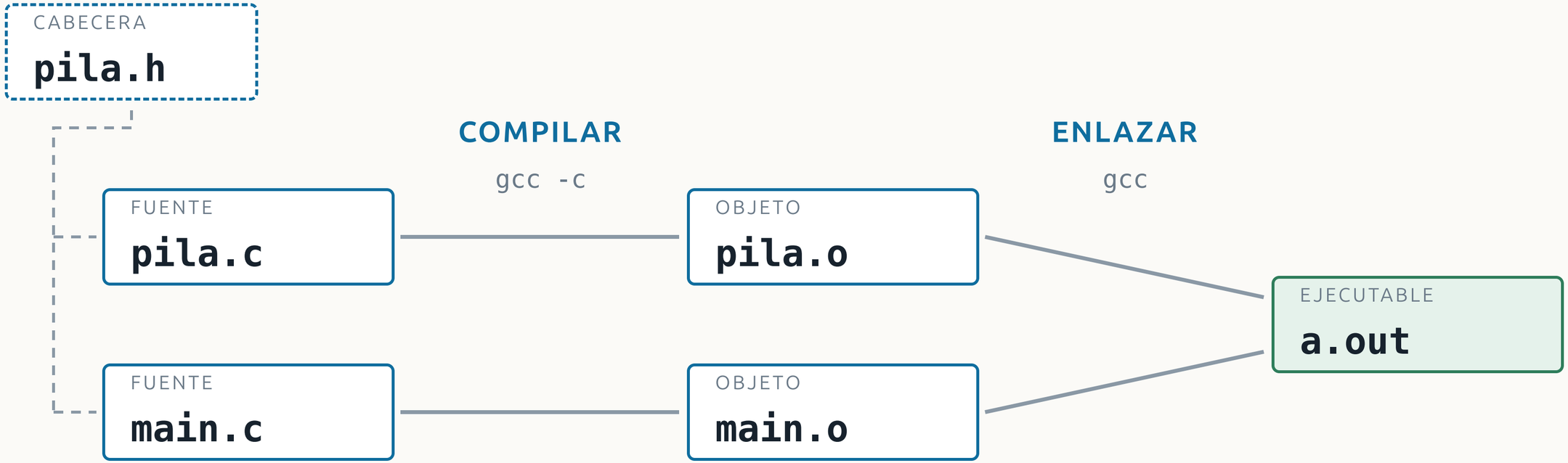

```
gcc -Wall -c pila.c
```

Entra un `.c`, sale un `.o`: el mismo código, pero en lenguaje máquina.

Quien traduce es el **compilador**. Lee **tu código**, y si no lo entiende, se queja señalando la línea.

Enlazar es juntar

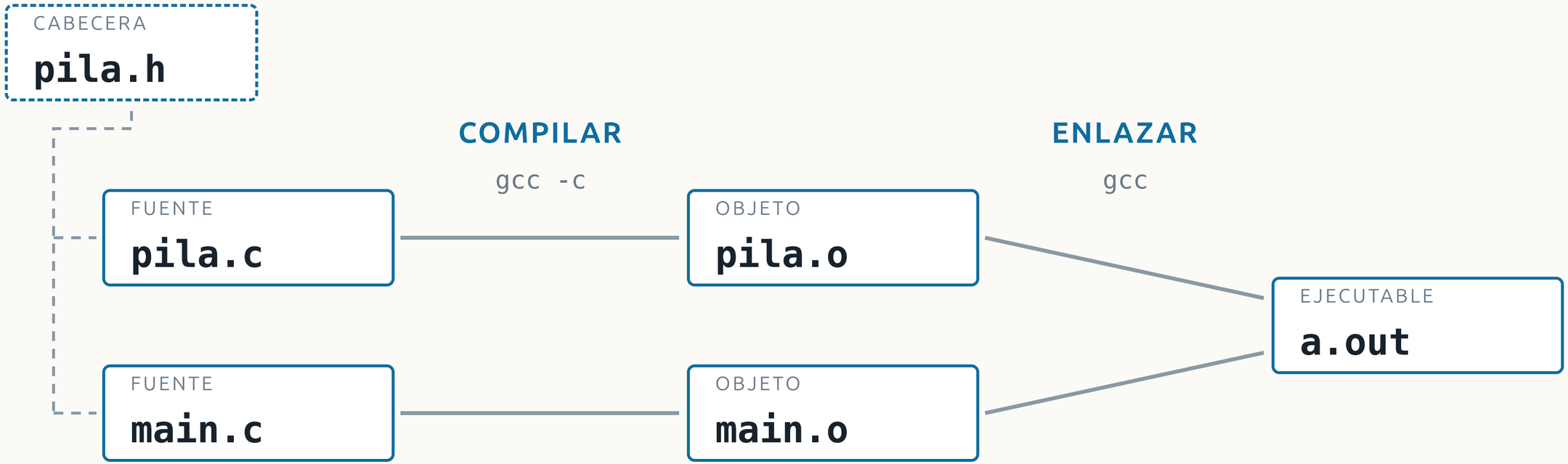
Los .o se cosen en un único ejecutable. Aquí se descubre si falta una pieza.



Quien junta es el **enlazador** (en los mensajes firma como `ld`). No lee tu código: solo comprueba que **cada función que se usa está en algún `.o`**. Lo llama gcc por ti.

```
gcc -Wall main.c pila.o
```

El dibujo completo



El `.h` no se compila nunca: se **copia dentro** de cada `.c` que lo incluye. Por eso va con línea discontinua.

Las dos órdenes que vais a usar

En las prácticas no vais a necesitar ninguna más.

pila.c → `gcc -Wall -c pila.c` → **pila.o**

Compila la librería. El -c dice: traduce y para, no intentes juntar nada.

main.c
pila.o → `gcc -Wall main.c pila.o` → **a.out**

Compila main.c y lo enlaza con el .o que ya tenías. Sale el ejecutable.

`-Wall` siempre. Sin él, gcc se calla la mitad de lo que sabe.

¿Qué hace el `-c`?

Es la opción que se olvida, y la que explica un error muy raro.

```
gcc -Wall -c pila.c
```

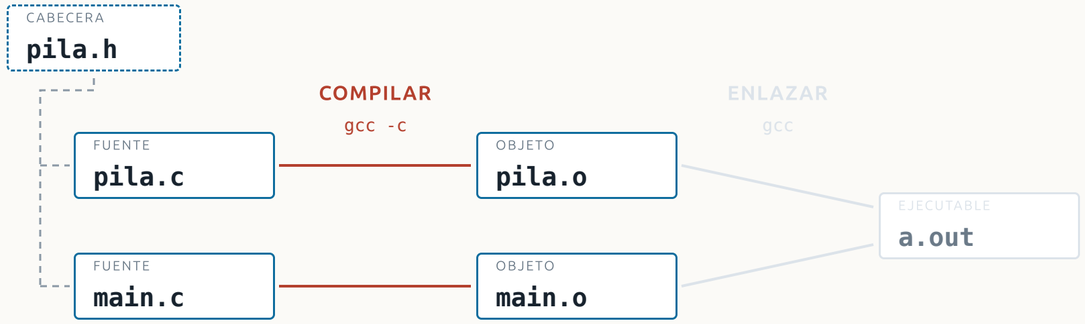
Con -c: traduce `pila.c` y **para**. Deja `pila.o` y no pregunta nada más.

```
gcc -Wall pila.c
```

Sin -c: traduce… y luego intenta **juntar** y hacer un ejecutable. Y busca un `main` que no existe.

```
undefined reference to 'main'
```

Cuando se queja el compilador

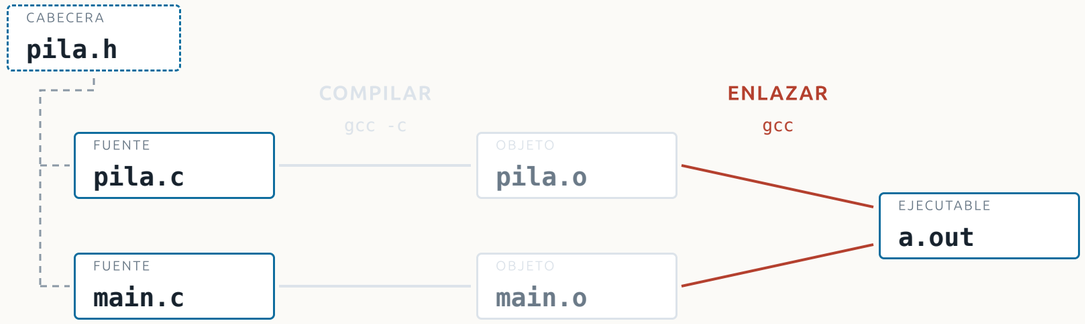



```
pila.c: In function 'apilar':
pila.c:12:5: error: 'nuevo' undeclared
   12 |     nuevo->sig = *p;
      |     ^~~~~
```

Fichero, línea, columna. Y debajo, tu línea con un acento donde está el problema.

Es **tu código**, en **esa línea**. Lee el mensaje entero: muchas veces dice la solución.

Cuando se queja el enlazador




```
/usr/bin/ld: main.c: in function 'main':
undefined reference to 'apilar'
undefined reference to 'cima'
collect2: error: ld returned 1 exit status
```

Sin número de línea, y lo firma `ld`. No es el compilador.

Tu código está bien. Falta un fichero en la orden: te has dejado el `pila.o`.

Un aviso es un error que aún no ha explotado

```c
int esMayorQueDiez(int x) {
    if (x > 10) {
        return 1;
    }
}

int main(void) {
    int n = 7;
    int resultado = esMayorQueDiez(n);
    int sinUsar;
    if (resultado = 1) {
        printf("%d es mayor que 10\n", n);
    } else {
        printf("%d no es mayor que 10\n", n);
    }
    char *nombre = "Ada";
    printf("Hola, %d\n", nombre);
}
```

LO QUE DEBERÍA IMPRIMIR

```
7 no es mayor que 10
Hola, Ada
```

GCC -WALL: COMPILA, CON CUATRO AVISOS

```
 5: warning: control reaches end of non-void function
10: warning: unused variable 'sinUsar'
11: warning: suggest parentheses around assignment
17: warning: format '%d' expects 'int',
             but argument is 'char *'
```

LO QUE IMPRIME DE VERDAD

```
7 es mayor que 10
Hola, 2063728681
```

Tres de los cuatro avisos eran fallos de verdad.
Compiló, se ejecutó, y mintió. Con `-Wall`, un aviso se arregla igual que un error.

Tres reglas que valen para el 90 % de los casos

1. **Lee solo el primer error.** Arréglalo, vuelve a compilar. Los demás suelen desaparecer solos.

2. **La línea es donde gcc se dio cuenta**, no siempre donde está el fallo. Si no ves nada raro, mira la línea de arriba.

3. **`undefined reference` = mira la orden**, no el código. Te falta un fichero.

El compilador no es un juez.
Es un lector.

Cuando se queja, te está diciendo exactamente en qué línea y por qué ha dejado de entenderte. Es la mejor pista que vas a tener.

Apuntes — el catálogo completo de mensajes

Ejercicios — nueve programas rotos, con comprobador